

에타빌더 백엔드 코드 리뷰

L/NKerthon 2026 심사 기준(주제 20 · 창의성 20 · 기술 30 · 실용성 20 · 발표 10)과 행사 규칙을 기준으로 본 리뷰

대상: GitHub main 376fbcd (10/2 08:29) · 범위: 백엔드 app/, ai_service.py, main.py, scripts/ 와 챗봇 관련 FE 수정 · 데이터 담당의 성적표·교양 영역 코드와 FE 팀원 코드는 연동 부분만 확인 · 작성: Claude Code

1. 한눈에 보기

서버 기능은 계획한 범위를 모두 구현했고, 실제 데이터(건국대학교 글로벌캠퍼스 학사요람 · 2026-2 강의계획서 · 에브리타임 강의평 통계)로 배포 서버에서 시연 흐름 12단계와 시연 대본 7문장을 검증했습니다. 배점이 가장 큰 기술적 구현(30점)에서 내세울 핵심은 **"AI는 말을 해석하고 판정만 한다. 시간표 배치는 알고리즘이 보장한다"**는 역할 분리입니다. 아래 다섯 가지는 발표 전에 정리하는 게 좋습니다.

#	심각도	내용	위치	소요
1	높음	/api/ask가 로그인 없이 팀 OpenAI 키로 아무 질문이나 답하는 공개 창구로 열려 있음 (React 화면은 쓰지 않음)	app/routers/chat.py:16	5분
2	중간	의존성 버전이 고정되지 않음 → 발표 직전 누가 푸시하면 Render가 최신 버전을 설치해 깨질 수 있음	requirements.txt	10분
3	규칙	행사 시작(10/1 19:30) 전 커밋 12개가 있음 (FastAPI-Gemini hello world, Streamlit 뼈대) → "사전 개발" 질문 대비 필요	git 기록	발표 1줄
4	중간	과목명이 여러 과목에 부분 일치하면 첫 과목을 말없이 고름 ("영어"→스크린영어, 후보 9개)	app/catalog.py:134	20분
5	낮음	비밀번호 최소 길이: 서버 4자, 화면 6자로 다름	app/schemas.py:114, web/src/components/AuthScreen.tsx:22	2분

이미 안전한 것 (배포 서버에서 직접 확인): SECRET_KEY 설정됨(기본값으로 위조한 토큰은 401 거부) · .env가 커밋된 이력 없음 · 비밀번호는 PBKDF2(10만 회)+계정별 salt · SQL은 모두 매개변수 바인딩 · 성적표 PDF는 저장하지 않고 학번·이름은 추출하지 않음.

2. 심사 기준별 평가

기준 (배점)	코드가 보여주는 것 (근거)	보완할 점	발표에서 말할 것
주제 적합성·문제 정의 (20)	졸업 요건(학사요람), 이번 학기 개설 강좌, 강의평을 한 번에 계산 (catalog.py, transcript.py, scheduler.py)	문제의 크기를 보여줄 숫자(설문)는 코드 밖에서 준비	학생이 오가는 4곳(요람·수강편람·에타·성적표)을 하나로. "시간표가 캠퍼스의 하루를 정한다"
창의성·차별성 (20)	말로 한 조건("실습 많은", "객관식 시험")을 체크리스트로 바꾸고 강의마다 판정 (insights.py) · 교수님별 추천, 과목 비교, 챗봇으로 바로 넣기 (answers.py)	기존 서비스와의 비교표 필요	에타 시간표·학교 수강신청과의 차이: 졸업 요건 + 수업 성향 + 대화
기술적 구현·완성도 (30)	백트래킹+비트마스크 생성기 · Structured Outputs · 데이터 근거 답변 · 2단 캐시(메모리+DB) · 호출별 시간 예산 · 실데이터 검증 · 배포 버전 표시	생성기·챗봇 자동 테스트 없음(테스트 23개는 성적표만) · 의존성 미고정 · 공개 /api/ask	구조도 + 숫자: 생성 0.1~1.3초, 판정 176강의 6.9초, 캐시 시 챗봇 0.5초
실용성·발전 가능성 (20)	실제 학사요람·강의계획서·강의평 통계, 성적표 PDF를 그대로 사용, 계정에 저장, 학과 추가는 데이터만	컴퓨터공학과만 지원 · 강의평 수집 방식(약관) 질문 가능	확장: requirements-catalog JSON만 추가(코드 수정 없음), 실서비스는 학교 강의평가 데이터 제휴

기준 (배점)	코드가 보여주는 것 (근거)	보완할 점	발표에서 말할 것
발표·질의응답 (10)	시연 대본 7문장 검증, 시연 점검 스크립트 scripts/demo_check.py	6분 구성, 대체 영상	9장 예상 질문 참고

3. 시스템 구조: 어디에 무엇을 썼나

요청 흐름: React + Vite 화면(web/) → HTTP JSON → FastAPI 서버(Render, main.py) → 아래 모듈 → OpenAI / Supabase

모듈	하는 일	AI
app/routers/*	API 17개 (auth, meta, transcript, timetable, chat, checklist, lectures, reviews)	–
app/scheduler.py	시간표 생성: 백트래킹 + 30분 비트마스크	없음
app/checklist.py	체크리스트 규칙·충족 판정	없음
app/nlu.py	챗봇: AI가 의도 추출 → 서버가 상태 합치기	해석
app/answers.py	데이터 근거 답변 · 과목 비교 · 과목 넣기	답변만
app/insights.py	자유 항목 판정 · 교수님별 비교	판정만
app/transcript.py	성적표: pypdf 텍스트 우선, 학교 형식이 아니면 AI	예비
ai_service.py	OpenAI API 호출 (gpt-6-luna, Structured Outputs), 캐시, 시간 예산	–
app/db.py	Supabase Postgres (계정, AI 응답 캐시, 수집 데이터)	–
data/seed/*.json	졸업 요건 · 2026-2 카탈로그 · 강의평 통계	–

파일 (줄)	역할	핵심 함수 / 클래스	쓴 기술
main.py (33)	라우터 등록, 상태 확인(배포 커밋 version)	health_check	FastAPI, CORS, StaticFiles
ai_service.py (251)	모든 OpenAI 호출 한 곳에 모음	ask_json, ask_text, _with_fallback, _cache_get/put, _db_get/put	OpenAI SDK(Structured Outputs, PDF-이미지 입력), SHA-256 캐시 키, LRU
app/scheduler.py (445)	시간표 생성기	Context, section_score, combo_score, search, pick_diverse, diagnose, generate	백트래킹, 비트마스크, heapq
app/checklist.py (249)	체크리스트 규칙과 충족 판정	apply_style, merge_item, lecture_level, syllabus_levels, evaluate_item, josa	규칙 기반
app/nlu.py (279)	챗봇 해석과 상태 합치기	ChatParse, _system_prompt, _parse, handle_chat	Structured Outputs
app/answers.py (346)	근거 답변, 과목 비교, 과목 넣기	answer, compare, compare_lectures, add_course_plan	OpenAI(답변만), 규칙
app/insights.py (321)	자유 항목 판정, 교수님별 비교	lecture_profile, judge_custom_items, on_demand	ThreadPoolExecutor, Structured Outputs
app/transcript.py (452)	성적표 → 이수 현황 (데이터 담당과 공동)	parse_pdf, parse_transcript_lines, summarize, ge_status	pypdf, OpenAI(예비)
app/catalog.py (184)	데이터 로딩(DB 우선, 없으면 파일)	Catalog.reload, find_course_by_name, category, is_offered	JSON, Postgres
app/auth.py (140)	회원가입·로그인·토큰	PgUserStore, issue_token, read_token	PBKDF2-SHA256, HMAC-SHA256
app/db.py (101)	Supabase 연결, 테이블 생성	init, execute, jsonb	psycopg 3
app/schemas.py (120)	API 요청-응답 형식	ChecklistItem, Conditions, GenerateRequest, ChatRequest, ChatContext	Pydantic v2

파일 (줄)	역할	핵심 함수 / 클래스	쓴 기술
scripts/demo_check.py (167)	시연 점검 12단계 + 캐시 채우기	step, chat, compare_chat, info_chat	표준 라이브러리만
tests/ (3파일, 데이터 담당)	성적표 읽기 테스트 23개	test_transcript_text/year/ge	pytest

4. AI를 쓰는 곳과 쓰지 않는 곳

AI가 하는 일	위치	언제 부르나	설정
챗봇 해석: 의도·조건·체크리스트·과목을 JSON으로	nlu._parse	챗봇 메시지마다	reasoning low · 20초×2 · DB 캐시
데이터 근거 답변 작성	answers.answer	정보 질문·잡담	low · 20초×2 · 메모리 캐시만(성적표 숫자 포함)
자유 항목 판정 (강의 10개씩 병렬)	insights._judge_chunk	체크리스트에 자유 항목이 있을 때	none · 25초×2 · DB 캐시
성적표 읽기(예비)	transcript.parse_pdf	학교 형식 PDF가 아닐 때	low · 40초×2 · 메모리 캐시만
수강평 분석	insights.extract_insights_from_texts	수집기 구현 후 (지금 미사용)	low · 15초×2
단순 채팅	routers/chat.py /api/ask	예전 Streamlit만	삭제 권장 (F1)

AI 없이 계산하는 것: 시간표 생성, 체크리스트 충족 판정, 졸업 요건·남은 학점, 교수님 순위, 과목 비교 요약, 과목 넣기 추천, 로그인. 그래서 같은 입력이면 같은 결과가 나오고, 시간 겹침·학점 초과가 생기지 않습니다.

5. 핵심 구현 상세 (기술 30점의 근거)

5.1 시간표 생성기 · app/scheduler.py

- **반드시 지키는 조건:** 시간 겹침 없음, 같은 과목은 한 분반, 공강 요일 수업 없음, 총 학점이 목표-2 이상 목표 이하, 고정한 분반 포함, 제한·이미 들은 과목 제외.
- **겹침 검사:** 요일×30분 칸을 정수 하나의 비트로 바꿔(section_mask) 두 분반이 겹치는지 AND 한 번으로 확인합니다.
- **탐색:** 점수가 높은 과목부터 넣거나 빼며 백트래킹(search). 남은 과목을 다 넣어도 최소 학점을 못 채우면 그 가지를 버립니다. 탐색 노드는 12만 개로 제한합니다.
- **다양한 상위 5개:** 과목 조합마다 가장 좋은 분반 배치 하나만 남기고(분반만 다른 변형 제거), 서로 2과목 이상 다른 조합을 고릅니다(pick_diverse).
- **조합이 없을 때(diagnose):** 고정 분반끼리 겹침·공강 충돌을 먼저 찾고, 조건을 하나씩 풀어 **실제로 다시 탐색해 조합이 생기는 방법만** 제안합니다.
- **성능:** 실데이터(127과목·312분반)에서 0.1~1.3초.

점수 항목	값	의도
남은 필수 과목	+20 (졸업 우선 스타일 ×1.5)	졸업 요건 먼저
영역 부족 학점	부족 학점/9 (최대 1) × 6, 학점 비례	비율이 아니라 부족 학점으로 급한 정도 판단
체크리스트 일치	과목마다 ±6	말한 성향 반영
대상 학년 차이	학년 차이 1마다 -1.5 (같으면 +1)	3학년에게 1학년 과목이 물리지 않게
에브리타임 별점	(별점-3.5)×1.5, 학점 비례	조건이 같으면 평이 좋은 강의
과목 수	과목마다 -1.5	1학점 과목을 여러 개 넣어 점수 불리기 방지
목표 학점 부족	1학점마다 -4	목표 학점에 가깝게

점수 항목	값	의도
개수형 조건(교양 2개 등)	1과목 차이마다 -30	사용자가 말한 개수 우선
대학 스타일	늦잠·동아리·취업·통학별 감점	스타일 프리셋

리뷰 의견: 결정적이고 설명 가능한 구조라 심사에서 강점입니다. 다만 노드 상한에 걸리면 최적을 보장하지 않는 근사 탐색이라는 점, 가중치가 손으로 정한 값이라는 점은 질문 대비가 필요합니다.

5.2 챗봇 · app/nlu.py → app/answers.py

- **3단계:** ① AI가 메시지에서 의도·바꿀 조건·체크리스트·과목을 정해진 JSON 형식으로 뽑음 → ② 서버가 규칙으로 상태를 합치고 “무엇을 바꿨는지” 문장을 만들 → ③ 질문이 섞이면 서버가 지금 시간표·졸업 현황·과목 정보를 모아 주고 AI가 그 안에서만 답함.
- **의도 6종:** set_preferences(조건), add_course(과목 넣기), course_review(교수님별 비교), compare_courses(두 과목 비교), ask_info(정보 질문), other(사용법·잡담).
- **장점:** 답장과 화면이 어긋나지 않음(“바꿨다”는 말은 서버 문장만), 데이터에 없으면 “지금 데이터에는 없어요”로 답해 지어내기를 막음, 판단할 수 없는 조건(“강의실 가까운”)은 반영 불가로 안내.
- **리뷰 의견:** 프롬프트가 코드 안에 있고(system_prompt, ANSWER_SYSTEM) 자동 회귀 테스트가 없어, 문구를 고칠 때 대표 문장 10개 정도를 다시 돌려 봐야 합니다.

5.3 체크리스트 · app/checklist.py

- **4종:** 기본(과제·팀플·시험·발표·출석, 레벨형), 시간표 모양(1교시·우주공강), 자유 항목(대화에서 생김, 레벨형/적용형), 개수형(교양 2개 등, 드롭다운).
- **판정:** 레벨형은 과목 평균(1.7 미만 적음, 2.3 초과 많음), 적용형은 반대 0개·맞음 1개 이상, 개수형은 개수 일치. 정보 없는 과목은 따로 표시.
- **수강계획서 보충:** 강의평에 없는 값은 계획서에서 사실로 읽히는 것만 씀(시험 횟수, 명시된 팀프로젝트·발표). 과제 비율은 강의평과 맞지 않아 제외(과제 0%인 강의도 강의평 70%가 ‘보통’).

5.4 자유 항목 판정 · app/insights.py

- **강의 정보 묶음(lecture_profile):** 에브리타임 통계, 시험 형식, 수강계획서 수업 방식·평가 비율, 이러닝 여부.
- **속도:** 강의 10개씩 나눠 최대 10개 동시 호출, 추론 강도 none, 결과는 DB에 캐시 → 176강의×6항목 6.9초, 두 번째부터 0초.
- **판정 가능 범위(실데이터 176강의):** 실습 비중 167 · 전자출결 68 · 객관식 시험 44 · 학점 잘 줌 43 · 온라인(이러닝) 7 · 교수님 친절함 0(원문 수강평 없음).

5.5 성적표 · app/transcript.py (데이터 담당과 공동)

- pypdf로 글자를 먼저 읽고(OpenAI 없이 0.4초), 학교 형식이 아니면 AI로 읽음. 입학년도 교육과정 기준으로 전필/전선을 나누고, F·N·NP·취득학점포기는 제외, 재수강은 마지막 기록만.
- 개인정보: PDF는 메모리에서만 읽고 버리며 학번·이름은 결과에 넣지 않음(가상 PDF 두 종류로 확인).
- 리뷰 의견: 학기 소계 줄이 없는 성적표에서는 “총 취득학점 37학점인데 읽은 과목은 0학점이에요”라는 잘못된 검산 경고가 뜸(F11).

5.6 AI 호출 계층 · ai_service.py / 5.7 계정·DB · app/auth.py, app/db.py

- 호출마다 시간 예산(챗봇 20초×2, 성적표 40초×2, 판정 25초×2)을 뒤 FE 대기 시간(45~90초) 안에 끝남. 인증 오류는 재시도 없이 바로 실패.
- 캐시 2단: 메모리 LRU 500개 + DB llm_cache(챗봇 해석·자유 항목 판정만). 성적표에서 나온 숫자가 들어가는 응답은 DB에 넣지 않음.
- Supabase 테이블 4개: app_users(계정·이수 내역), seed_data(데이터 교체용), collected_insights(수집 강의평), llm_cache. DB가 끊기면 메모리·파일로 계속 동작.

6. 검증 결과

항목	결과	방법
시연 흐름	12단계 모두 정상	scripts/demo_check.py, 배포 서버
시연 대본 7문장	순서대로 모두 정상 (교양 2개 충족 포함)	배포 서버 da1324d
성적표 테스트	23 passed	pytest, 데이터 담당 작성
학교 형식 성적표(가상)	OpenAI 없이 0.4초, 입학년도 2023↔2025에서 전필/전선 9/15→3/21	DB·OpenAI 끄고 로컬 서버
한글 성적표(가상, AI 경로)	25과목 모두 정확, 개인정보 0	fpdf2로 만든 2쪽 PDF
시간표 생성	0.1~1.3초	실데이터
챗봇 응답	2~5초, 캐시 시 0.5초	배포 서버
보안 점검	기본 SECRET_KEY 토큰 401 거부 · /api/ask 공개 확인	배포 서버 직접 요청

7. 상세 발견 사항

ID	심각도	위치	문제와 영향	고치는 법
F1	높음	app/routers/chat.py:16 POST /api/ask	로그인 없이 임의 문장을 OpenAI로 전달. 저장소가 공개라 주소와 사용법이 드러나 있어 누구나 팀 키로 쓸 수 있음 → 비용·한도 소진, 시연 중 429 위험.	React 화면은 쓰지 않음(예전 frontend/app.py만). 라우트 삭제 또는 로그인 필수로.
F2	중간	requirements.txt	openai>=2.0처럼 범위만 지정. 재배포 때 새 버전이 깔려 예고 없이 깨질 수 있음(발표 직전 팀원 푸시도 재배포).	로컬에서 검증한 버전으로 고정: fastapi==0.128.8, uvicorn==0.39.0, openai==2.48.0, pydantic==2.13.5, python-multipart==0.0.20, psycopg[binary]==3.2.13, pypdf==6.19.0, python-dotenv==1.2.1 (Render에서 1회 배포 확인).
F3	규칙	git 기록 (9/26~10/1 00:13)	행사 시작 전 커밋 12개: FastAPI-Gemini hello world, Streamlit 뼈대, 503 재시도. 규칙 "핵심 소스코드 사전 준비 불가"에 대한 질문이 나올 수 있음.	사전 커밋은 환경 구축(허용) 수준이고 Gemini 기반이며, 행사 중 OpenAI로 바꾸고 전부 다시 작성했음. 발표에 "핵심 기능은 행사 중 개발, 커밋 기록으로 확인 가능"을 명시.
F4	중간	app/catalog.py:134 find_course_by_name	부분 일치하면 첫 과목을 반환. "영어 넣어줘"→스크린영어(후보 9), "글쓰기"→글쓰기심화(5), "프로그래밍"→C++ 프로그래밍(8). 사용자가 원한 것과 다른 과목을 넣을 수 있음.	후보가 2개 이상이면 과목 넣기·비교에서 후보 버튼으로 되돌기(이미 있는 선택지 카드 재사용).
F5	중간	/api/chat 등 AI 호출 API	요청 수 제한이 없어 반복 호출 시 OpenAI 비용이 쌓임.	시연 규모에선 낮은 위험. IP별 분당 제한 정도 권장.
F6	중간	tests/	자동 테스트가 성적표에만 있음. 생성기·챗봇은 시연 점검 스크립트로만 확인.	생성기 제약(겹침·학점·공강·고정) 테스트 5개만 있어도 '완성도' 근거가 됨.
F7	낮음	app/schemas.py:114 AuthScreen.tsx:22	비밀번호 최소 길이 서버 4, 화면 6.	서버도 6으로.
F8	낮음	app/everytime.py:11	fetch_reviews 미구현. 강의평 없는 교수님은 '수강계획서 기준' 또는 '정보 없음'.	발표에서는 "강의평 통계 배치 수집"으로 설명(실시간 수집이라고 하지 않기).
F9	낮음	데이터	강의평 원문 없이 통계만 있어 '교수님 친절함' 같은 항목은 판정 불가.	시연 대본에서 제외(완료).

ID	심각도	위치	문제와 영향	고치는 법
F10	낮음	app/db.py:84	DB 연결 1개를 전역 잠금으로 공유 → 동시 요청이 많으면 DB 대기.	시연 규모에선 문제없음. 실서비스면 연결 풀.
F11	낮음	app/transcript.py_verify	학기 소개 줄이 없으면 “읽은 과목은 0학점” 잘못된 경고.	소개가 없을 때 전체 과목 합과 비교(데이터 담당).
F12	정보	web/src/components/GradPanel.tsx	교양 영역 묶음을 SHOW_GE_AREAS=false로 숨김(팀 결정). 서버 계산은 그대로.	데이터 담당과 공유.

8. 발표에서 반드시 밝힐 외부 자료 (행사 규칙 7쪽)

규칙: “외부 코드나 오픈소스 데이터를 활용했다면 최종 발표에서 주요 활용 내용을 반드시 밝힐 것.” 발표 마지막 장에 아래 표를 그대로 넣는 것을 권합니다.

구분	내용	쓴 곳
생성형 AI (서비스 기능)	OpenAI API · gpt-6-luna (Chat Completions, Structured Outputs, PDF 입력)	챗봇, 자유 항목 판정, 성적표 예비
생성형 AI (개발 보조)	Claude Code (커밋에 Co-Authored-By 기록) — 규칙상 허용, 밝히는 것을 권장	백엔드·챗봇 코드 작성 보조
백엔드 오픈소스	FastAPI, Uvicorn, Pydantic, OpenAI Python SDK, pycopg, pypdf, python-dotenv, python-multipart	서버 전반
프론트 오픈소스	React, Vite, TypeScript	화면
외부 서비스	Supabase(Postgres), Render(배포), GitHub	DB, 배포, 협업
데이터	건국대학교 글로벌캠퍼스 학사요람(졸업 요건·교육과정 2023~2026), 2026-2 강의계획서, 에브리타임 강의평 통계(87개 강의)	졸업 요건, 카탈로그, 강의 성향

9. 발표 자료 필수 구성(행사 규칙 9쪽)에 넣을 내용

필수 구성	넣을 내용
문제 정의 및 주제 연관성	시간표를 짜려면 학사요람·수강편람·에타·성적표를 오가야 함 → 졸업 요건 실수, 수업 성향 확인에 시간 소모. “시간표가 캠퍼스의 하루를 정한다.”
아이디어 및 해결 방안	성적표 한 장으로 남은 졸업 요건 계산 + 대화로 말한 성향을 강의평·계획서로 판정 + 알고리즘이 조건을 지키는 시간표 5개 추천
주요 기능 및 구현 방법	3장 구조도, 4장 AI 역할 분리, 5.1 생성기(비트마스크·백트래킹), 5.2 챗봇 3단계, 기술 스택
결과물 및 시연	시연 대본: ① 금요일 공강·팀플 적게 ② 교양 과목 2개 ③ 시스템프로그래밍 넣기 ④ KUGEP1 교수님 추천 ⑤ 데이터베이스·컴퓨터네트워크 비교 ⑥ 남은 졸업 학점 질문 ⑦ 강의실 가까운(반영 불가)
기대 효과 및 활용 방안	다른 학과·학교는 졸업 요건·개설 강좌 JSON만 추가(코드 그대로) · 강의평은 학교 강의평가 데이터 제휴로 전환 · 수강신청 장바구니 연동

10. 예상 질문과 답변

질문	답변 요지
LLM이 시간표를 직접 짜지 않는 이유는?	시간 검침·학점 초과 같은 제약을 확률적으로 어길 수 있어서. 제약은 알고리즘이 보장하고 AI는 말 해석과 수강평 판정만 맡김. 같은 입력이면 같은 결과.
AI가 지어낸 값 아닌가?	판정·답변은 서버가 넘긴 데이터만 근거로 하고 없으면 ‘정보 없음’. 체크리스트와 상세창에 출처(강의평/수강계획서) 표시.

질문	답변 요지
에브리타임 데이터는 어떻게 모았고 약관은?	강의평 통계를 배치로 모아 저장, 요청마다 긁지 않음. 실서비스에서는 학교 강의평가 데이터 제휴로 바꿀 계획.
개인정보는?	성적표 PDF는 저장하지 않고 학번·이름은 추출하지 않음. 계정에는 과목 목록만. 비밀번호는 해시 저장.
다른 학과·학교로 확장하려면?	졸업 요건·개설 강좌 JSON만 추가하면 코드 수정 없이 동작. 드롭다운의 '준비 중' 학과가 그 구조.
비용과 속도는?	같은 입력은 메모리·DB 캐시로 재사용, 판정은 병렬. 생성 0.1~1.3초, 챗봇 2~5초(캐시시 0.5초).
행사 전에 만든 코드가 있나?	서버 hello world·배포 설정·Streamlit 뼈대(환경 구축)만. 핵심 기능은 10/1 19:30 이후 개발했고 커밋 기록으로 확인 가능.